



# CTD Python Essentials

Week 12: Advanced Data Visualization techniques

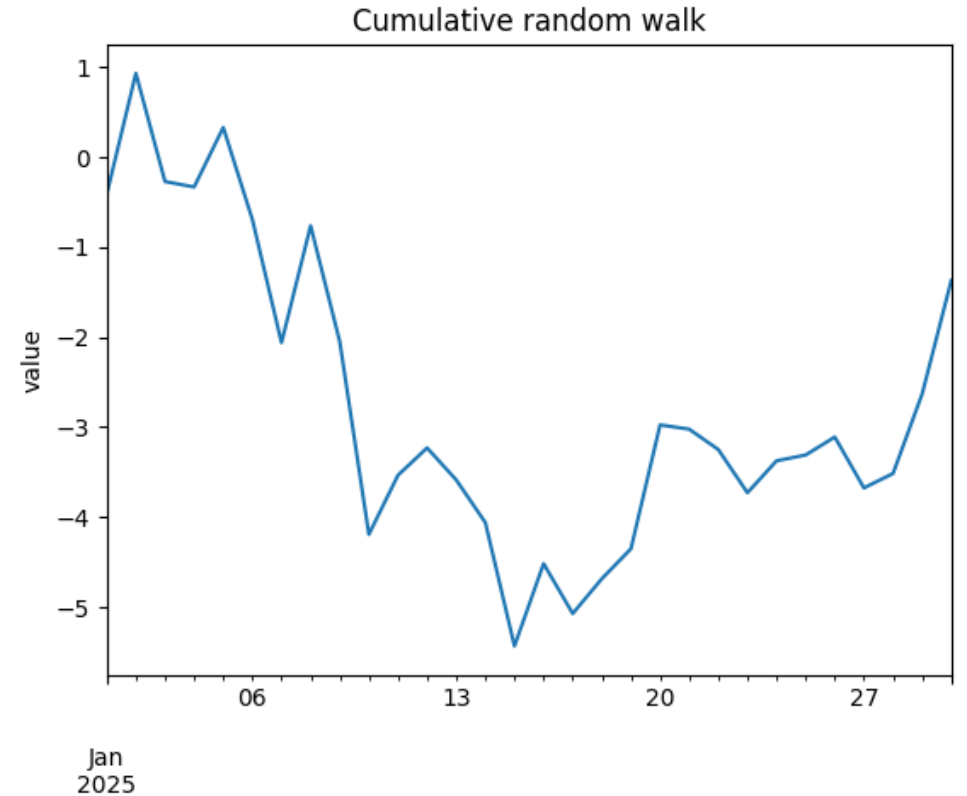


# Plotting in Pandas

```
# a tiny synthetic "sensor" signal
idx = pd.date_range("2025-01-01", periods=30, freq="D")
signal = pd.Series(np.random.randn(len(idx)).cumsum(), index=idx)

signal.plot(title="Cumulative random walk", ylabel="value")

# When your DataFrame/Series is already indexed by DatetimeIndex,
# pandas takes care of the x-axis formatting and tick spacing for you.
```

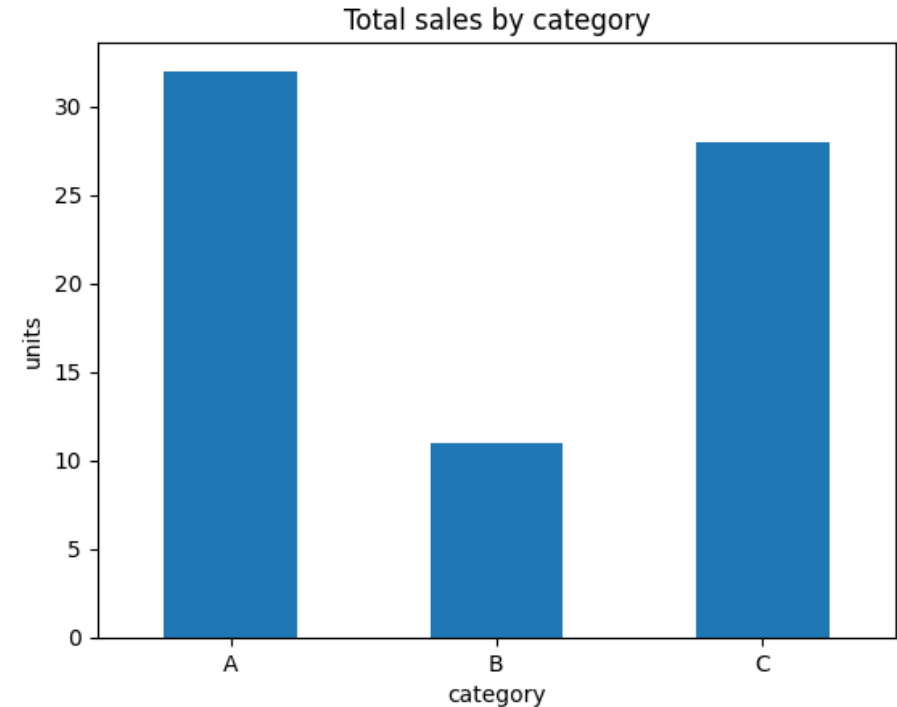


- A pandas line plot example
  - When your DataFrame/Series is already indexed by [DatetimeIndex](#), pandas takes care of the x-axis formatting and tick spacing for you.

# Plotting with Pandas

```
df = pd.DataFrame({
    "category": ["A", "B", "A", "C", "B", "A", "C"],
    "sales":     [12, 7, 9, 15, 4, 11, 13],
})

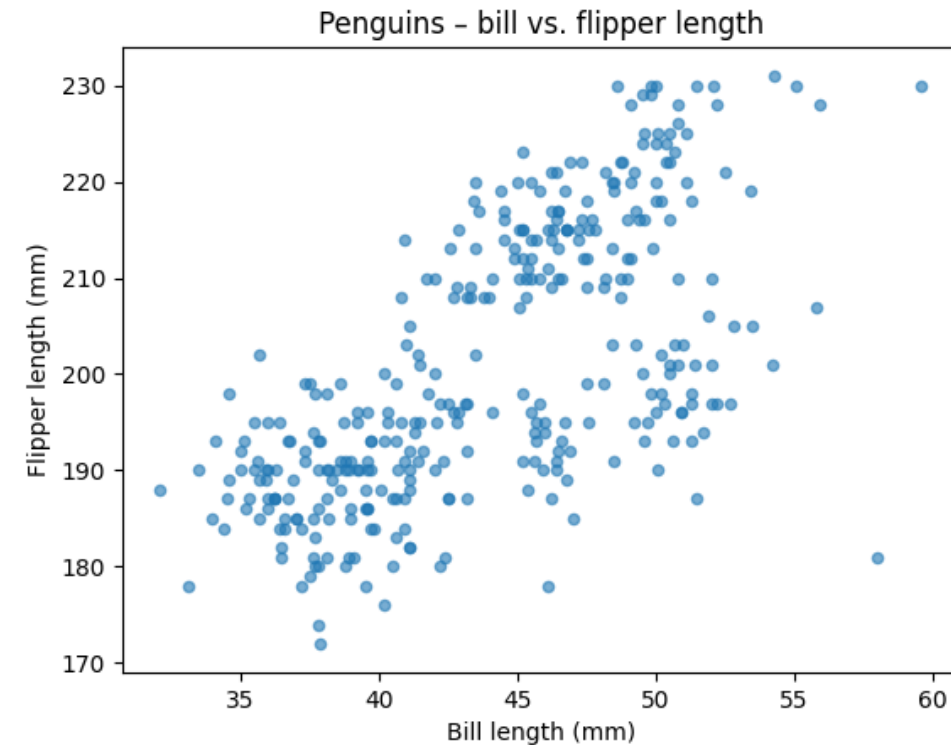
(df
 .groupby("category")["sales"]
 .sum()
 .plot(kind="bar", rot=0, title="Total sales by category", ylabel="units"))
```



- Pandas bar chart from grouped data
  - Because the Series you get from `groupby().sum()` already has category labels as its index, pandas automatically uses them as bar labels.

# Plotting with Pandas

```
penguins = pd.read_csv(  
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv"  
)  
  
(penguins  
    .dropna(subset=["bill_length_mm", "flipper_length_mm"]) # keep complete cases  
    .plot.scatter(x="bill_length_mm", y="flipper_length_mm",  
                 alpha=0.6, title="Penguins - bill vs. flipper length",  
                 xlabel="Bill length (mm)", ylabel="Flipper length (mm)"))
```



- Scatter plot from the *Palmer Penguins* dataset
  - `.plot.scatter()` lets you work directly with the DataFrame columns—no need to extract NumPy arrays first. Dropping the missing rows is a one-liner too, so the whole workflow stays in pandas.

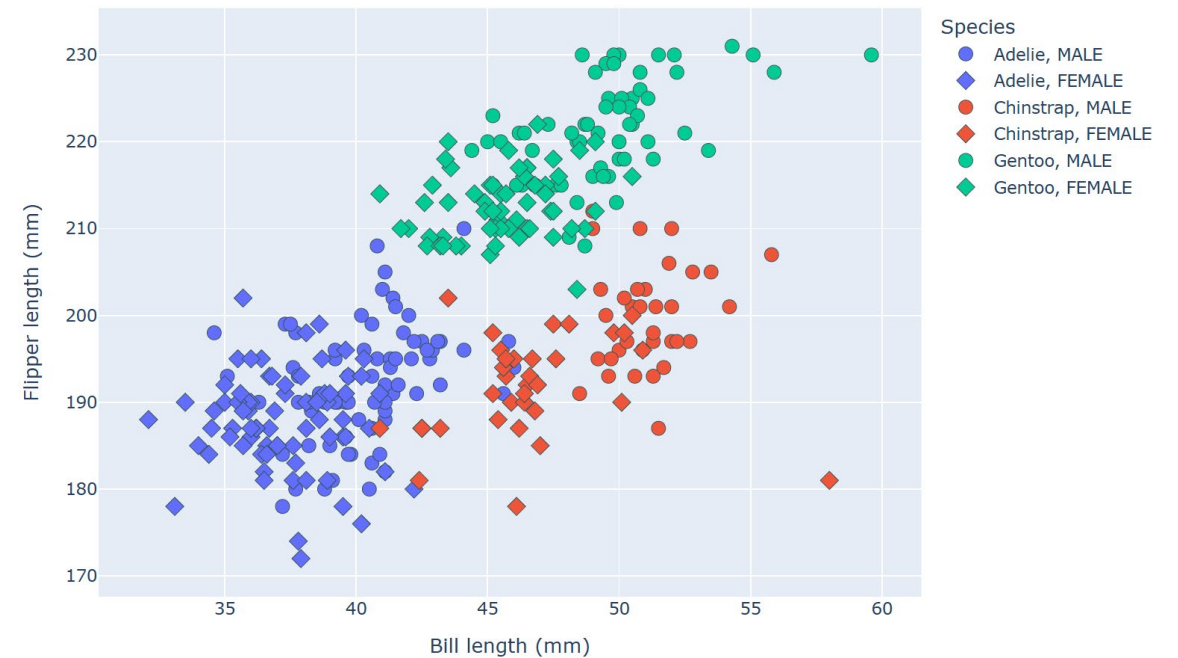
# Using Plotly for interactive Visualizations

```
import pandas as pd
import plotly.express as px
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv"
penguins = (
    pd.read_csv(url)
    # keep rows that have the columns we want for x, y, and aesthetics
    .dropna(subset=["bill_length_mm", "flipper_length_mm", "species", "sex"])
)

fig = px.scatter(
    penguins,
    x="bill_length_mm",
    y="flipper_length_mm",
    color="species",
    symbol="sex",
    hover_data=["island", "body_mass_g"],
    title="Palmer Penguins: Bill Length vs Flipper Length",
    labels={
        "bill_length_mm": "Bill length (mm)",
        "flipper_length_mm": "Flipper length (mm)",
        "body_mass_g": "Body mass (g)"
    },
    height=600,
    width=850
)

fig.update_traces(marker=dict(size=10, line=dict(width=0.5, color="DarkSlateGrey")))
fig.update_layout(legend_title_text="Species")
fig.write_html("iris.html", auto_open=True)
```

Palmer Penguins: Bill Length vs Flipper Length

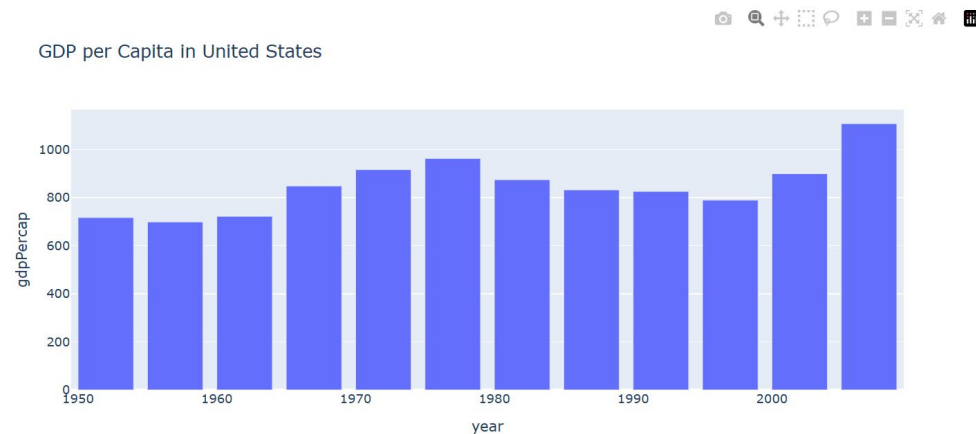
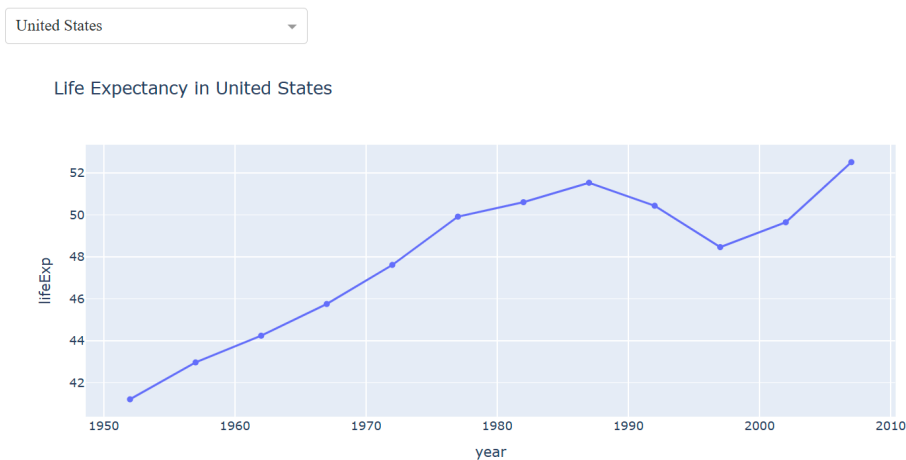


- This is an image of interactive scatter plot of the Palmer Penguins dataset
  - Run the code to try it. It will pop up in your browser.
  - The code is available as a [Github gist](#)

# Dynamic Dashboards using Plotly and Dash

- Allows creation of interactive web applications which update in real time.
- Launches a web server
- Here's the [code](#) as a github gist
  - Server at <http://127.0.0.1:8050>
- Another example: [live crypto dashboard](#)

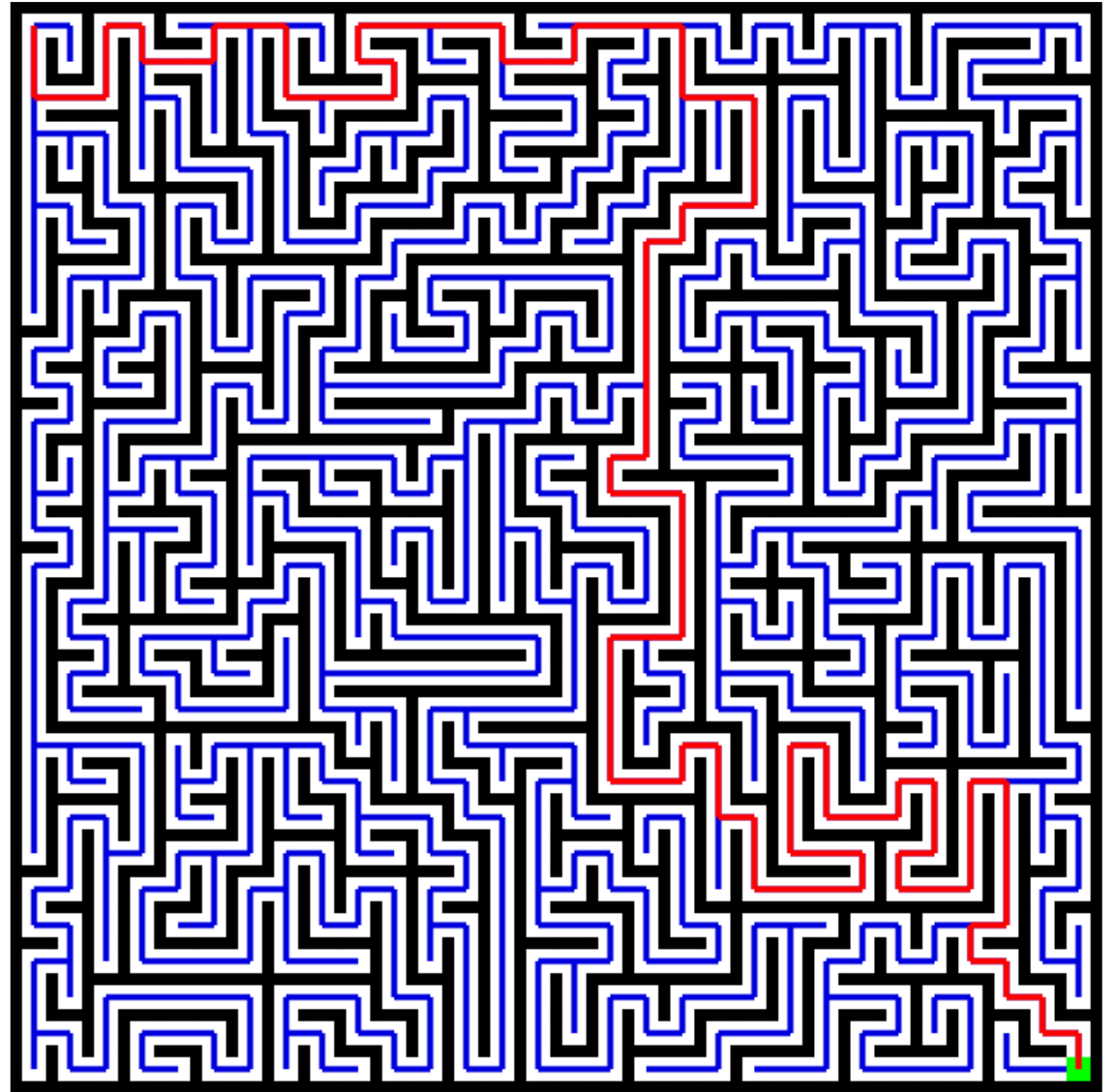
## Gapminder Country Dashboard



# Flask

---

- Lightweight python framework which is useful for creating APIs and simple servers.
- Dash is built on [Flask](#)
- An example of a simple Flask server
  - [Generate and solve mazes using reinforcement learning](#)
  - [Demo](#)





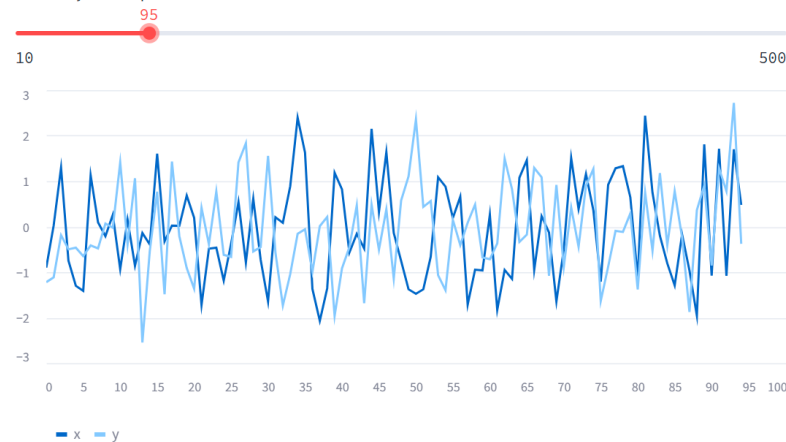
# Streamlit

- An alternative to Dash which is straightforward to set up and use
  - Here's the example code in a [gist](#)
  - command line: `streamlit run streamlit_3min.py`

## 3-Minute Streamlit Demo

Pick a sample size with the slider and watch the chart update *instantly*— no callbacks, HTML, or JavaScript required.

How many random points?



Done! That's the entire app — fewer than 20 lines including imports.

```
import streamlit as st
import numpy as np
import pandas as pd

st.title("🕒 3-Minute Streamlit Demo")

st.write(
    """
    Pick a sample size with the slider and watch the chart update *instantly*—
    no callbacks, HTML, or JavaScript required.
    """
)

n = st.slider("How many random points?", min_value=10, max_value=500, value=50)

data = pd.DataFrame(
    np.random.randn(n, 2),
    columns=["x", "y"],
)

st.line_chart(data)

st.success("Done! That's the entire app — fewer than 20 lines including imports.")
```

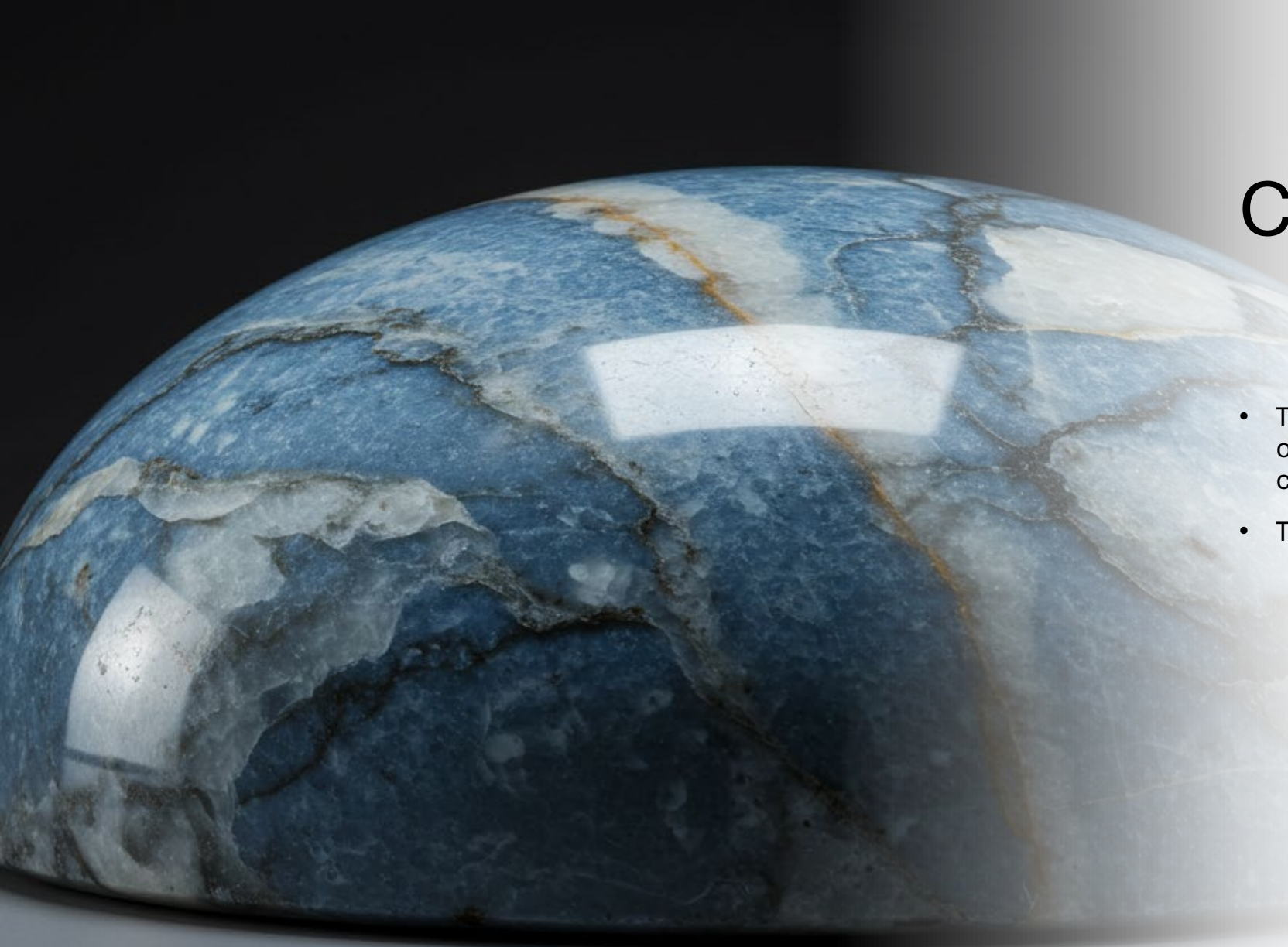


# Contrasting Dash and Streamlit

- Streamlit
  - fast and simple
  - Automatic reactivity and state
  - Auto refresh
  - Quick prototypes / last-minute presentations
- Dash/Plotly
  - Fine grained configurability, scales to complex multi-page apps
  - Explicit reactivity and state
  - Fine grained event driven can be more efficient
  - Production dashboards

# Publishing Dashboards

- There are convenient ways to publish Dashboard for both Dash and Streamlit
- For Dash we recommend [Render.com](https://render.com)
  - Create a service by publishing a public github repo
- For Streamlit you can deploy for free on [streamlit.io](https://streamlit.io)
- For the assignment this week, you will publish the url's for these services in [service\\_urls.txt](#) in the assignment12 folder



# Capstone Project

- The final three weeks will provide an opportunity to use skills from the class to create capstone projects
- Two projects
  - Kaggle based data pipeline
    - Student choose one of four dataset and perform cleaning, aggregation, analysis and presentation
    - Alternatively use another dataset with CIL approval
  - Local project published on github
    - Scraping, persistence, analysis and publication in a dashboard
    - Use either Dash or Streamlit

# Lesson 12 Infrastructure

- It's different than past lessons and assignments
- The lesson uses a new repo [python-assignment12](#)
  - Separate from any other repos
  - It's used to deploy to [render.com](#)
- If you do the optional Streamlit lesson, there's another independent repo for that called [streamlit-assignment](#)
  - Used to deploy to [streamlit.io](#)
- Summary:
  - A pull request for [assignment12](#) is pasted into the first link submission field in the homework submission form. If you do the [streamlit-assignment](#), the link for that repository is pasted into the second link field on the assignment submission form. You create and commit a file inside the assignment12 folder called [service\\_urls.txt](#). This file contains a URL for your render.com service. If you do the Streamlit lesson, add a second URL for streamlit.io service.



# Q&A and Demo

